Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)
КАФЕДРА «Системы обработки информации и управления» (ИУ5)

ОТЧЕТ
по лабораторной работе № 1

«Основные конструкции языка Python»

по дисциплине
«АСОИУ»

Выполнил:
студент группы ИУ5-33Б _____ Тютюнник Н.В.
(подпись)
«___» _____ 2026 г.

Проверил:
преподаватель кафедры ИУ5 _____ Гапанюк Ю.Е.
(подпись)
«___» _____ 2026 г.

2026

1 Цель работы

Изучение базовых конструкций языка программирования Python: функций, классов, условных операторов и циклов, обработки исключений, работы с аргументами командной строки, а также оператора сопоставления с образцом (match/case).

2 Задание

Разработать программы для решения биквадратного уравнения. Требования к программам:

а) программы должны быть консольными приложениями на языке Python. Необходимо разработать три программы в различных стилях:
   - процедурном – с использованием функций (оператор def);
   - объектно-ориентированном – с использованием классов;
   - функциональном – с использованием сопоставления с образцом (pattern matching);

б) программа осуществляет ввод с клавиатуры коэффициентов A, B, C, вычисляет дискриминант и находит действительные корни уравнения в зависимости от значения дискриминанта;

в) коэффициенты A, B, C могут быть заданы в виде параметров командной строки. Если коэффициенты не заданы, они вводятся с клавиатуры в соответствии с пунктом б);

г) если коэффициент A, B или C введён или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно до тех пор, пока он не будет введён корректно. Корректно заданный коэффициент – это коэффициент, значение которого может быть без ошибок преобразовано в действительное число.

3 Описание решения

3.1 Метод решения биквадратного уравнения

Биквадратным называется уравнение вида

$$Ax^4 + Bx^2 + C = 0, \quad (1)$$

где $A \neq 0$. Заменой $y = x^2$ уравнение (1) сводится к вспомогательному квадратному уравнению

$$Ay^2 + By + C = 0, \quad (2)$$

дискриминант которого равен

$$D = B^2 - 4AC. \quad (3)$$

В зависимости от значения дискриминанта возможны следующие случаи:

- $D < 0$ – уравнение (2) не имеет действительных корней, следовательно, уравнение (1) также не имеет действительных корней;
- $D = 0$ – уравнение (2) имеет один корень $y = -B / (2A)$;
- $D > 0$ – уравнение (2) имеет два корня $y_{1,2} = (-B \pm \sqrt{D}) / (2A)$.

Для каждого найденного корня y находятся корни исходного уравнения: при $y > 0$ – два корня $x = \pm\sqrt{y}$, при $y = 0$ – один корень $x = 0$, при $y < 0$ действительных корней нет. Таким образом, биквадратное уравнение может иметь от нуля до четырёх действительных корней. Повторяющиеся корни исключаются, итоговый список корней выводится в порядке возрастания.

Дополнительно обрабатываются вырожденные случаи: при $A = 0$ уравнение принимает вид $Bx^2 + C = 0$ и имеет решение $x^2 = -C / B$; при $A = B = 0$ и $C = 0$ корнем является любое действительное число, а при $C \neq 0$ уравнение решений не имеет.

3.2 Ввод и проверка коэффициентов

Коэффициент с номером i (i = 1, 2, 3) сначала ищется в списке аргументов командной строки sys.argv. Если аргумент отсутствует или не может быть преобразован в действительное число функцией float(), выводится

сообщение об ошибке и коэффициент запрашивается с клавиатуры. Ввод повторяется до получения корректного значения. Значения nan и inf, которые функция float() принимает без ошибок, также считаются некорректными, так как с ними невозможно вычислить корни.

3.3 Структура программ

Все три программы реализуют одинаковый алгоритм и выдают одинаковый результат, но написаны в разных стилях.

Программа `biquadratic_procedural.py` (процедурный стиль) состоит из набора функций: parse_coef – преобразование строки в число, get_coef – чтение коэффициента, get_discriminant – вычисление дискриминанта, get_square_roots – решение уравнения $x^2 = y$, get_roots – вычисление всех корней, print_roots – вывод результата, main – основная функция.

Программа `biquadratic_oop.py` (объектно-ориентированный стиль) содержит два класса. Класс CoefReader отвечает за чтение коэффициентов из командной строки и с клавиатуры. Класс BiquadraticEquation хранит коэффициенты уравнения, вычисляет дискриминант (свойство discriminant), находит корни (метод roots) и выводит решение (метод print_solution); строковое представление уравнения формируется методом __str__.

Программа `biquadratic_functional.py` (функциональный стиль) построена на чистых функциях, возвращающих значения без изменения состояния. Ветвление выполняется оператором сопоставления с образцом match/case: по значению дискриминанта, по кортежу коэффициентов (A, B, C), по длине среза аргументов командной строки и по количеству найденных корней. Повторный ввод реализован рекурсией, объединение корней – функцией reduce, преобразование и фильтрация – функциями map и filter и лямбда-выражениями.

4 Текст программ

Листинг 1 – Процедурный стиль (файл biquadratic_procedural.py)

```
import sys
import math

MSG_BAD_ARG = 'Некорректное значение в командной строке: "{}"'
MSG_BAD_INPUT = 'Некорректное значение: "{}". Повторите ввод.'


def parse_coef(coef_str):
    try:
        coef = float(coef_str)
    except ValueError:
        return None
    if not math.isfinite(coef):
        return None
    return coef


def get_coef(index, prompt):
    if index < len(sys.argv):
        coef = parse_coef(sys.argv[index])
        if coef is not None:
            return coef
        print(MSG_BAD_ARG.format(sys.argv[index]))
    while True:
        coef_str = input(prompt)
        coef = parse_coef(coef_str)
        if coef is not None:
            return coef
        print(MSG_BAD_INPUT.format(coef_str))


def get_discriminant(a, b, c):
    return b * b - 4 * a * c


def get_square_roots(y):
    if y > 0.0:
        sq_y = math.sqrt(y)
        return [-sq_y, sq_y]
    if y == 0.0:
        return [0.0]
    return []


def get_roots(a, b, c):
    y_list = []
    if a == 0.0:
        if b != 0.0:
            y_list.append(-c / b)
    else:
        d = get_discriminant(a, b, c)
        if d == 0.0:
            y_list.append(-b / (2.0 * a))
        elif d > 0.0:
```

```
            sq_d = math.sqrt(d)
            y_list.append((-b + sq_d) / (2.0 * a))
            y_list.append((-b - sq_d) / (2.0 * a))
    result = []
    for y in y_list:
        result.extend(get_square_roots(y))
    return sorted(set(result))



def format_number(x):
    return '{:.6g}'.format(x)



def equation_to_str(a, b, c):
    sign = lambda x: '-' if x < 0 else '+'
    return '{}*x^4 {} {}*x^2 {} {} = 0'.format(
        format_number(a), sign(b), format_number(abs(b)),
        sign(c), format_number(abs(c)))



def print_roots(roots):
    len_roots = len(roots)
    if len_roots == 0:
        print('Действительных корней нет')
    elif len_roots == 1:
        print('Один корень: {}'.format(format_number(roots[0])))
    else:
        words = {2: 'Два корня', 3: 'Три корня', 4: 'Четыре корня'}
        print('{}: {}'.format(words[len_roots],
                              ', '.join(format_number(r) for r in roots)))



def main():
    a = get_coef(1, 'Введите коэффициент A: ')
    b = get_coef(2, 'Введите коэффициент B: ')
    c = get_coef(3, 'Введите коэффициент C: ')
    print('Уравнение:', equation_to_str(a, b, c))

    if a == 0.0 and b == 0.0:
        if c == 0.0:
            print('Корнем является любое действительное число')
        else:
            print('Уравнение не имеет решений')
        return

    if a == 0.0:
        print('A = 0: уравнение вырождается в B*x^2 + C = 0')
    else:
        d = get_discriminant(a, b, c)
        print('Дискриминант D = {}'.format(format_number(d)))
        if d < 0.0:
            print('D < 0: вспомогательное уравнение '
                  'не имеет действительных корней')
        elif d == 0.0:
            print('D = 0: вспомогательное уравнение имеет один корень')
        else:
            print('D > 0: вспомогательное уравнение имеет два корня')

    print_roots(get_roots(a, b, c))
```

```
if __name__ == "__main__":
    main()
```

Листинг 2 – Объектно-ориентированный стиль (файл biquadratic_oop.py)

```
import sys
import math

MSG_BAD_ARG = 'Некорректное значение в командной строке: "{}"'
MSG_BAD_INPUT = 'Некорректное значение: "{}". Повторите ввод.'


class CoefReader:
    def __init__(self, argv):
        self.argv = argv

    @staticmethod
    def parse(coef_str):
        try:
            coef = float(coef_str)
        except ValueError:
            return None
        if not math.isfinite(coef):
            return None
        return coef

    def read(self, index, prompt):
        if index < len(self.argv):
            coef = self.parse(self.argv[index])
            if coef is not None:
                return coef
            print(MSG_BAD_ARG.format(self.argv[index]))
        while True:
            coef_str = input(prompt)
            coef = self.parse(coef_str)
            if coef is not None:
                return coef
            print(MSG_BAD_INPUT.format(coef_str))


class BiquadraticEquation:
    ROOT_WORDS = {2: 'Два корня', 3: 'Три корня', 4: 'Четыре корня'}

    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    @staticmethod
    def fmt(x):
        return '{:.6g}'.format(x)

    def __str__(self):
        sign = lambda x: '-' if x < 0 else '+'
        return '{}*x^4 {} {}*x^2 {} {} = 0'.format(
            self.fmt(self.a), sign(self.b), self.fmt(abs(self.b)),
```

```
        sign(self.c), self.fmt(abs(self.c)))

    @property
    def discriminant(self):
        return self.b * self.b - 4 * self.a * self.c

    @property
    def is_identity(self):
        return self.a == 0.0 and self.b == 0.0 and self.c == 0.0

    def _y_roots(self):
        if self.a == 0.0:
            return [-self.c / self.b] if self.b != 0.0 else []
        d = self.discriminant
        if d < 0.0:
            return []
        if d == 0.0:
            return [-self.b / (2.0 * self.a)]
        sq_d = math.sqrt(d)
        return [(-self.b + sq_d) / (2.0 * self.a),
                (-self.b - sq_d) / (2.0 * self.a)]

    @staticmethod
    def _x_roots(y):
        if y > 0.0:
            sq_y = math.sqrt(y)
            return [-sq_y, sq_y]
        if y == 0.0:
            return [0.0]
        return []

    def roots(self):
        result = set()
        for y in self._y_roots():
            result.update(self._x_roots(y))
        return sorted(result)

    def print_solution(self):
        print('Уравнение:', self)
        if self.is_identity:
            print('Корнем является любое действительное число')
            return
        if self.a == 0.0 and self.b == 0.0:
            print('Уравнение не имеет решений')
            return
        if self.a == 0.0:
            print('A = 0: уравнение вырождается в B*x^2 + C = 0')
        else:
            d = self.discriminant
            print('Дискриминант D = {}'.format(self.fmt(d)))
            if d < 0.0:
                print('D < 0: вспомогательное уравнение '
                      'не имеет действительных корней')
            elif d == 0.0:
                print('D = 0: вспомогательное уравнение имеет один корень')
            else:
                print('D > 0: вспомогательное уравнение имеет два корня')

        roots = self.roots()
        if len(roots) == 0:
            print('Действительных корней нет')
```

```
        elif len(roots) == 1:
            print('Один корень: {}'.format(self.fmt(roots[0])))
        else:
            print('{}: {}'.format(self.ROOT_WORDS[len(roots)],
                                  ', '.join(self.fmt(r) for r in roots)))


def main():
    reader = CoefReader(sys.argv)
    a = reader.read(1, 'Введите коэффициент A: ')
    b = reader.read(2, 'Введите коэффициент B: ')
    c = reader.read(3, 'Введите коэффициент C: ')
    BiquadraticEquation(a, b, c).print_solution()


if __name__ == "__main__":
    main()
```

Листинг 3 – Функциональный стиль с сопоставлением с образцом (файл biquadratic_functional.py)

```
import sys
import math
from functools import reduce

MSG_BAD_ARG = 'Некорректное значение в командной строке: "{}"'
MSG_BAD_INPUT = 'Некорректное значение: "{}". Повторите ввод.'


def parse_coef(coef_str):
    try:
        coef = float(coef_str)
    except ValueError:
        return None
    match coef:
        case x if math.isfinite(x):
            return x
        case _:
            return None


def ask_coef(prompt):
    coef_str = input(prompt)
    match parse_coef(coef_str):
        case None:
            print(MSG_BAD_INPUT.format(coef_str))
            return ask_coef(prompt)
        case coef:
            return coef


def get_coef(argv, index, prompt):
    match argv[index:index + 1]:
        case []:
            return ask_coef(prompt)
        case [coef_str]:
            match parse_coef(coef_str):
                case None:
```

```
                    print(MSG_BAD_ARG.format(coef_str))
                    return ask_coef(prompt)
                case coef:
                    return coef


def discriminant(a, b, c):
    return b * b - 4 * a * c


def x_roots(y):
    match y:
        case 0:
            return (0.0,)
        case y if y > 0:
            return (-math.sqrt(y), math.sqrt(y))
        case _:
            return ()


def y_roots(a, b, d):
    match d:
        case 0:
            return (-b / (2.0 * a),)
        case d if d > 0:
            return ((-b + math.sqrt(d)) / (2.0 * a),
                    (-b - math.sqrt(d)) / (2.0 * a))
        case _:
            return ()


def all_x_roots(ys):
    roots = reduce(lambda acc, y: acc | set(x_roots(y)), ys, set())
    return tuple(sorted(roots))


def solve(a, b, c):
    match (a, b, c):
        case (0, 0, 0):
            return ('identity', None, ())
        case (0, 0, _):
            return ('contradiction', None, ())
        case (0, _, _):
            return ('degenerate', None, all_x_roots((-c / b,)))
        case _:
            d = discriminant(a, b, c)
            return ('biquadratic', d, all_x_roots(y_roots(a, b, d)))


fmt = lambda x: '{:.6g}'.format(x)

sign = lambda x: '-' if x < 0 else '+'

equation_to_str = lambda a, b, c: '{}*x^4 {} {}*x^2 {} {} = 0'.format(
    fmt(a), sign(b), fmt(abs(b)), sign(c), fmt(abs(c)))

ROOT_WORDS = {2: 'Два корня', 3: 'Три корня', 4: 'Четыре корня'}
```

```
def describe_case(kind, d):
    match (kind, d):
        case ('identity', _):
            return 'Корнем является любое действительное число'
        case ('contradiction', _):
            return 'Уравнение не имеет решений'
        case ('degenerate', _):
            return 'A = 0: уравнение вырождается в B*x^2 + C = 0'
        case ('biquadratic', 0):
            return ('Дискриминант D = 0\n'
                    'D = 0: вспомогательное уравнение имеет один корень')
        case ('biquadratic', d) if d > 0:
            return ('Дискриминант D = {}\n'
                    'D > 0: вспомогательное уравнение имеет два корня'
                    ).format(fmt(d))
        case ('biquadratic', d):
            return ('Дискриминант D = {}\nD < 0: вспомогательное уравнение '
                    'не имеет действительных корней').format(fmt(d))



def describe_roots(kind, roots):
    match (kind, roots):
        case ('identity' | 'contradiction', _):
            return None
        case (_, ()):
            return 'Действительных корней нет'
        case (_, (x,)):
            return 'Один корень: {}'.format(fmt(x))
        case (_, roots):
            return '{}: {}'.format(ROOT_WORDS[len(roots)],
                                   ', '.join(map(fmt, roots)))



def main():
    a, b, c = map(lambda arg: get_coef(sys.argv, *arg),
                  ((1, 'Введите коэффициент A: '),
                   (2, 'Введите коэффициент B: '),
                   (3, 'Введите коэффициент C: ')))
    kind, d, roots = solve(a, b, c)
    lines = ('Уравнение: ' + equation_to_str(a, b, c),
             describe_case(kind, d),
             describe_roots(kind, roots))
    print('\n'.join(filter(None, lines)))



if __name__ == "__main__":
    main()
```

5 Результаты выполнения программ

На рисунках 1–6 приведены примеры запуска программ в терминале с различными наборами коэффициентов, охватывающими все варианты значения дискриминанта, вырожденные случаи и обработку некорректного ввода.

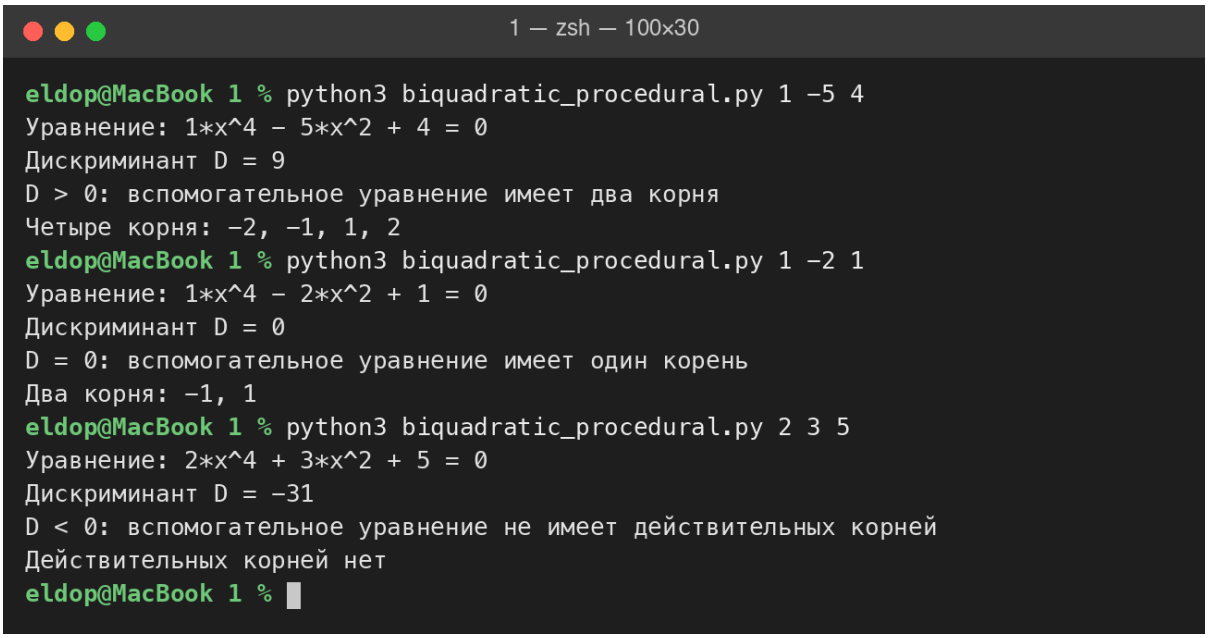


Рисунок 1 – Процедурная программа: коэффициенты заданы в командной строке (D > 0, D = 0, D < 0)

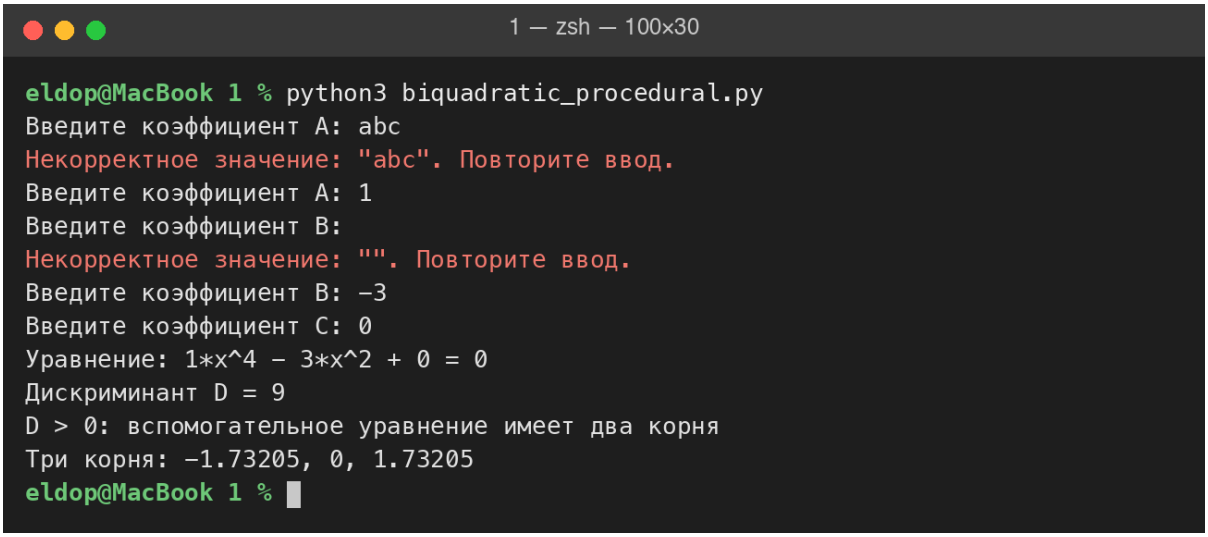


Рисунок 2 – Процедурная программа: ввод коэффициентов с клавиатуры с повтором при ошибке

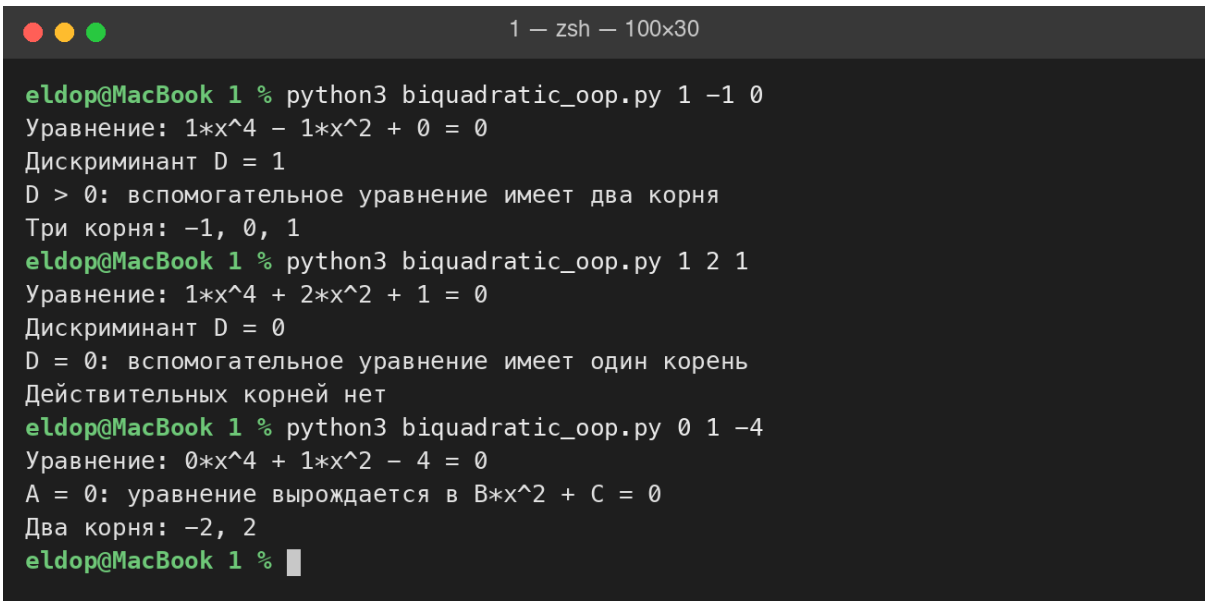


Рисунок 3 – Объектно-ориентированная программа: три корня, отсутствие корней при D = 0, случай A = 0

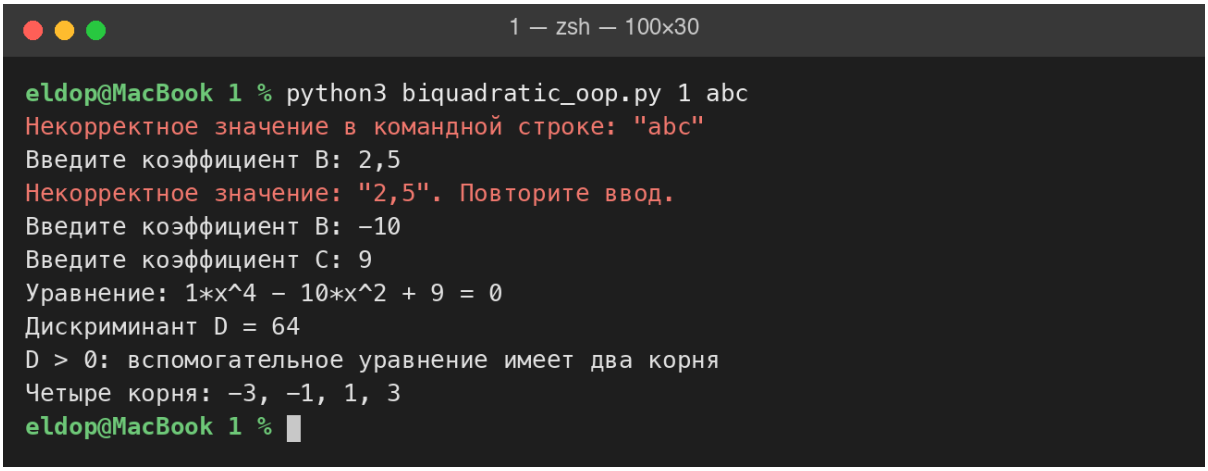


Рисунок 4 – Объектно-ориентированная программа: некорректный аргумент командной строки

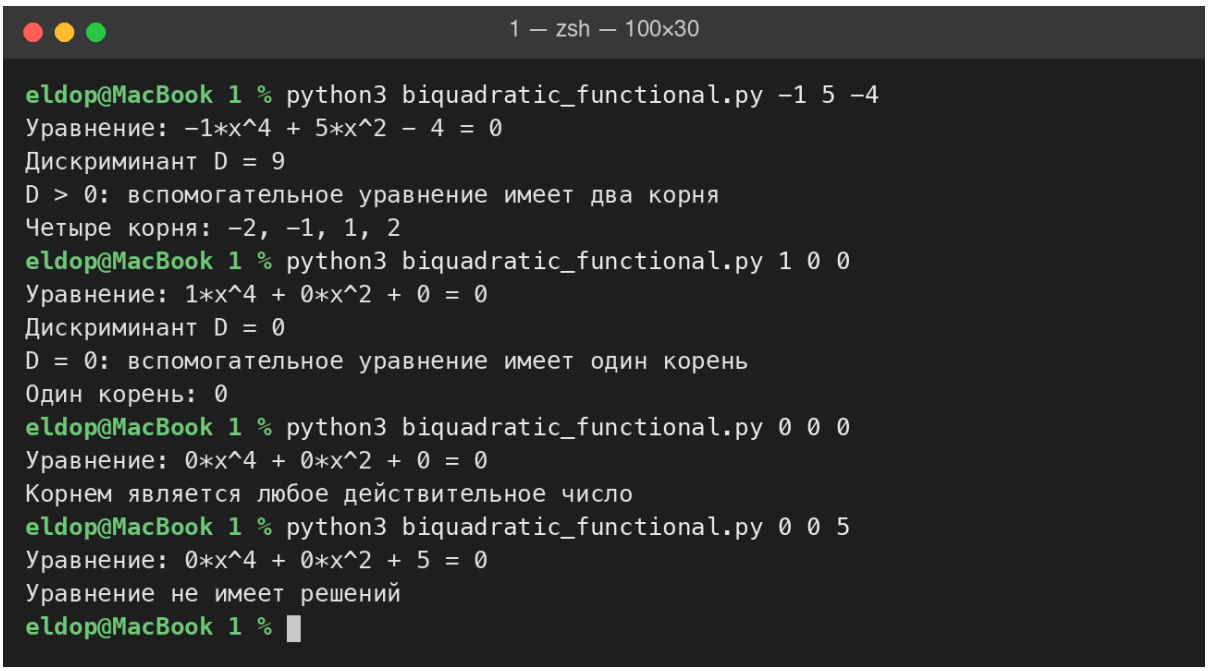


Рисунок 5 – Функциональная программа: четыре корня, один корень и вырожденные случаи

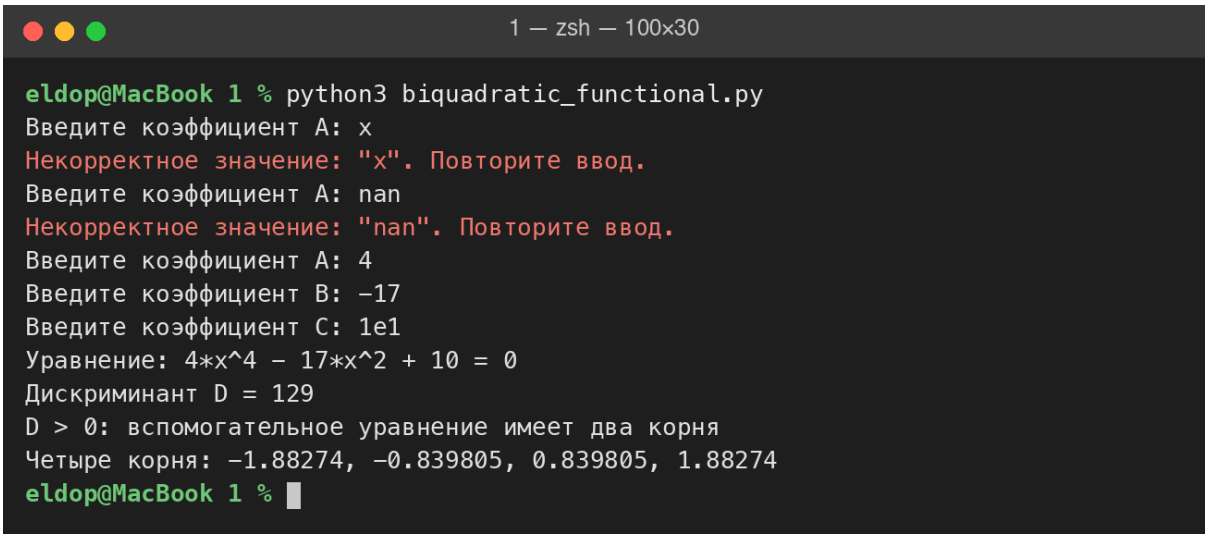


Рисунок 6 – Функциональная программа: ввод с клавиатуры с некорректными значениями

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены основные конструкции языка Python. Разработаны три консольные программы для нахождения действительных корней биквадратного уравнения, реализованные в процедурном, объектно-ориентированном и функциональном стилях.

Программы поддерживают задание коэффициентов как через аргументы командной строки, так и с клавиатуры, корректно обрабатывают ошибочный ввод, все варианты знака дискриминанта и вырожденные случаи. Результаты работы трёх программ совпадают на всех рассмотренных наборах коэффициентов.

Сравнение стилей показало, что процедурный стиль наиболее прост и близок к записи алгоритма, объектно-ориентированный позволяет объединить данные уравнения и операции над ними в одном классе, а функциональный стиль с оператором match/case делает разбор случаев компактным и наглядным.